Semantic 3D Reconstruction and Object Tracking for Transparent Objects

Lex Byrne
byrne296@umn.edu

Vinh Nguyen
nguy4766@umn.edu

I. Introduction and Motivation

Reconstructing and tracking transparent objects, such as glassware or plastic containers, presents unique challenges in computer vision. Traditional 3D reconstruction methods, such as depth cameras, struggle with these objects, as they distort keypoint detection and depth estimation. We present a pipeline for detecting and tracking transparent objects while reconstructing a 3D scene and estimating their 6D poses.

This work is particularly motivated by robotic detection and grasping of transparent objects. Unlike general AR or VR applications where visual realism is prioritized, we focus on ensuring accurate object detection, localization, and pose estimation for robotic manipulation. Addressing these challenges can improve robotic pick-and-place tasks, warehouse automation, and assistive robotics.

II. Related Work

Several methods have been proposed for reconstructing transparent objects. Light transport in transparent objects is much more complex than in opaque objects. Multi-view stereo (MVS) methods are based on matching points across views, which can fail if the appearance of those points changes drastically across views [3]. Strictly geometric approaches thus cannot handle general 3D reconstruction of transparent objects outside of controlled environments. Furthermore, sensors such as depth cameras often fail to gauge the depth of transparent objects [11].

Similarly, some newer methods modify existing techniques to handle transparent objects by explicitly modeling their complicated physical properties. For instance, Deng et al. [6] augment a neural 3D surface reconstruction technique by calculating how refractive surfaces deflect rays. However, humans can usually gauge the shape of transparent objects without perceiving the surrounding geometry. Hence, another approach is to learn the properties of transparent objects from data. 2D learning methods, which can leverage the abundance of 2D image data, may be used to guide 3D reconstruction. For instance, deep neural networks trained on 2D images can predict depth maps from only a single image [13] [10]. Li et al. [9] combine the two approaches by training a network with a differentiable layer that models refraction.

Because of the diversity of transparent objects, a model that learns about transparent objects from data must draw on a wide range of transparent objects. To this end, several large-scale datasets of transparent objects have been created. Mei *et al.* [5] present a glass detection dataset with ground-truth masks of glass areas. However, this focuses on large-scale glass objects and does not contain more general information. Xie *et al.* [4] present Trans10K and Trans10K-v2, which contain many types of transparent objects categorized by their use, with an emphasis on robotics applications. However, we elect to use the ClearPose dataset [2], described below.

Two papers stand out as especially relevant to this project:

A. ClearGrasp: 3D Shape Estimation of Transparent Objects for Manipulation

ClearGrasp [1] presents a deep learning-based framework designed to improve depth estimation for transparent objects. Since standard RGB-D sensors often fail to provide accurate depth information for transparent surfaces, ClearGrasp employs a convolutional neural network (CNN) trained to predict surface normals and depth corrections. The system consists of three primary components: (1) an occlusion boundary detection network, (2) a surface normal prediction network, and (3) a depth refinement module that integrates the corrected depth predictions into a more accurate 3D representation of the object.

ClearGrasp uses a synthetic dataset for training, generated via physically-based rendering techniques to simulate transparent object interactions realistically. While

ClearGrasp's method has proven effective in robotic manipulation tasks, we use a different dataset based on actual transparent objects to ensure our scenario is as realistic as possible.

B. ClearPose: Large-scale Transparent Object Dataset and Benchmark

ClearPose [2] introduces a large-scale dataset dedicated to transparent object perception. While previous datasets for transparent objects tended to be smaller or more specialized, ClearPose provides over 350K labeled real-world RGB-D frames and more than 5 million instance annotations across 63 household transparent objects. It contains scenes with multiple transparent objects, which are realistic and more challenging than scenes with only one.

The dataset is structured to support multiple tasks, including segmentation, depth completion, and 6D object pose estimation. In addition to its dataset contribution, ClearPose introduces several baseline models that evaluate the performance of current deep learning techniques on transparent object perception. The benchmarking framework provided by ClearPose enables direct comparison of novel algorithms against established methods, making it an essential reference for evaluating the effectiveness of our proposed approach.

ClearPose serves as the primary benchmark for our project, allowing us to rigorously assess segmentation accuracy, depth prediction improvements, and pose estimation performance. The dataset's scale and diversity ensure that our method generalizes well to real-world applications.

III. Proposed Approach

A. Dataset

We use the ClearPose dataset. Specifically, we focus on Set 9, Scene 11 of the dataset, which places the objects on a challenging non-planar surface.

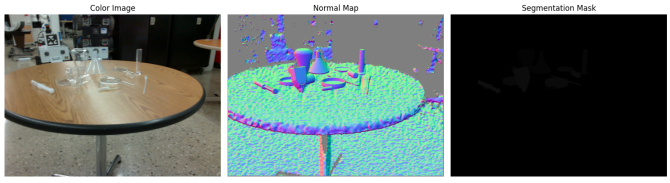

Fig. 1. Sample RGB image, ground truth normal map, and segmentation mask from the ClearPose dataset

B. Object Detection and Segmentation

1) Segmentation with SAM2 Using Bounding Box Prompting: We use the Segment Anything Model, specifically SAM2 [7], for segmentation tasks. SAM2 is accurate in segmenting a single transparent object in an image but struggles if there are many, which is the case for most of the scenes in our dataset. By providing bounding box prompts, SAM2 can focus on specific regions, improving segmentation accuracy.

Fig. 2. Segmentation results of SAM2 without bounding box prompting

Fig. 3. Segmentation results of SAM2 with bounding box prompting

2) Object Detection and Tracking: For SAM2 to work correctly for the dataset, we train an object detection model on our dataset to obtain the bounding boxes of transparent objects. The chosen approach is to fine-tune the pre-trained YOLOv8 model on our data set with the ultralytics Python package. YOLOv8 excels at one-shot detection, which is appropriate for a real-time robotic system.

For object tracking, we tested the DeepSORT tracking algorithm on a sample video from the dataset; however, there were persistent failures. Instead of DeepSORT, we implement our own approach by extracting the visual embeddings from each bounding box region and matching their similarities across camera angles.

C. 3D Reconstruction

1) 3D Structure from Motion with VGGSfM: We use VGGSfM [8], a pre-trained model that improves

the traditional SfM pipeline. This method can offer significant benefits when reconstructing scenes involving transparent objects, as it relies on learning as well as geometry, allowing it to handle occlusions, reflections, and refractions better than conventional SfM methods. In contrast, COLMAP relies only on geometry and SIFT features, which can fail when faced with transparent objects and specular effects. Like COLMAP, VGGSfM returns camera poses along with a sparse 3D reconstruction of the scene.

2) 3D Segmentation: To transfer the 2D segmentation masks to 3D, we project the masks to 3D using the camera poses from VGGSfM. One obstacle with this approach is that point clouds cannot easily detect occlusion, so occluded background points may be erroneously included in the projected mask. To mitigate this problem, all segmentation masks for an object are combined, and a 3D point is only included as part of an object if enough camera views contain the point within their masks.

Lifting the segmentation to 3D also enables further filtering and deduplication. If point clouds strongly overlap, their objects are considered to be duplicates, and their segmentation masks are combined. After this, if an object is detected in too few camera views, it is considered to be an outlier and discarded.

3) 6D Pose Estimation: We use segmented 3D points to estimate the 6D pose of an object. In a real deployment scenario, we imagine that the device reconstructing the scene has foreknowledge of the true 3D model of the object. For instance, a robot could record the 3D models of common objects it encounters, or even generate 3D models based on the segmentation mask. For our purposes, however, we manually assign known 3D point clouds of objects from ClearPose to point cloud segments in order to evaluate the remainder of our pipeline.

Because the scale of each object is unclear, we use Scale-ICP [14] to estimate both scale and 6D pose. Specifically, we attempt to align the known 3D point cloud of each object with its segmented reconstructed points. For initial scale and pose, we use the 3D bounding boxes around both point clouds to estimate scale and translation, and run Scale-ICP 10 times on each object with random initial rotations. Since Scale-ICP, like ICP, is a local algorithm, using different initial rotations is crucial to success.

IV. Experiments and Results

A. Object Detection with YOLOv8

1) Data Preprocessing: To construct an effective object detector for transparent objects, we began with the ClearPose dataset. The raw data set organizes color images and per-image metadata (including bounding box labels) across multiple sets and scenes. For ease of use and compatibility with YOLO-formatted object detection models, a preprocessing script was developed to automate the following tasks:

- The color images were copied into a dedicated directory with unique file names that concatenate set, scene, and frame identifiers.
- The information of the ground-truth bounding box was extracted from the provided `.mat` files and reformatted as axis-aligned rectangles in the `(xmin, ymin, xmax, ymax)` pixel coordinates. These were stored in a CSV file.

The bounding box annotations were then converted into YOLO format (`class x_center y_center width height`, normalized by image dimensions), and split into training, validation, and test subsets using a stratified split (60% train, 20% validation, 20% test).

2) Model Training: We adopted the YOLOv8n model, the smallest neural network in the YOLOv8 family, for its balance of speed and accuracy, making it practical for both experimentation and deployment in robotics contexts. Training was performed using the Ultralytics YOLO library on a Colab instance with a NVIDIA T4 GPU.

A sample of 500 images was allocated for preliminary experiments due to resource constraints. The model was initialized from publicly available pretrained weights (`yolov8n.pt`), and trained for 100 epochs with a resolution of 640×640 pixels. All samples were labeled with a single class, corresponding to "transparent object".

3) Evaluation and Results: YOLO's built-in evaluation metrics were used, including the mean average precision (mAP) at the IoU (Intersection over Union) threshold of 50% (mAP@0.5) and the overall precision. Visualizations of the validation results, confusion matrices, and qualitative sample predictions were generated to assess the practical viability of the model.

In the held-out test split, the trained YOLOv8n model achieved an accuracy of approximately 94%. The detected bounding boxes were sharp and consistently aligned with ground truth regions, and the network exhibited high-confidence predictions even for objects with minimal feature contrast—a common challenge in transparent object detection. The inference speed was sufficient for use in real time, which is critical for robotics applications. Qualitative examination confirmed that, despite a relatively limited training set,

the model generalized well across different objects and backgrounds in ClearPose.

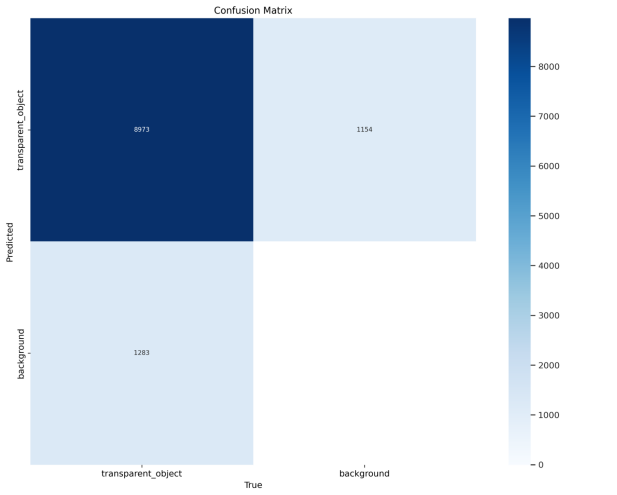

Fig. 4. Confusion matrix for predicting transparent object from static images

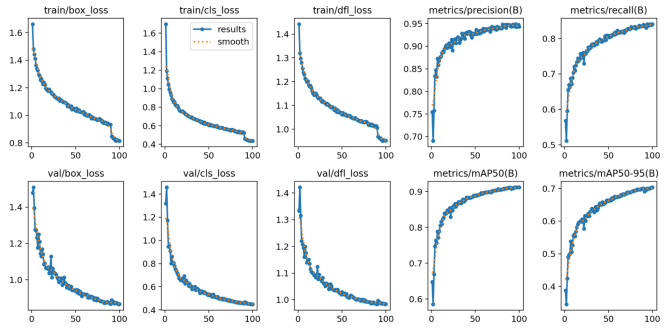

Fig. 5. Training curves for YOLOv8 prediction model

B. Segmentation with SAM2 and Bounding-Box Prompts

1) SAM2 Fine-Tuning: Segmentation of transparent objects presents significant challenges for conventional methods, so we leveraged the recently released Segment Anything Model 2 (SAM2) as the backbone for this task. Initial experiments with unprompted segmentation using SAM2, where no prompts such as points or bounding boxes were provided, yielded incomplete instance masks. The model occasionally failed to delineate object contours accurately or confused object boundaries with visual background clutter, especially in cluttered or low-contrast scenes typical for transparent objects.

To address this, we fine-tuned SAM2 on a subset of the ClearPose dataset. Instance masks in ClearPose were converted into COCO-style polygon annotations for

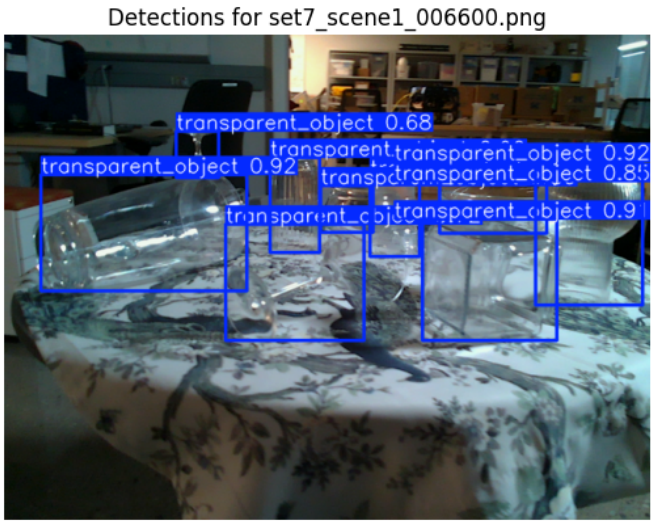

Fig. 6. Example detection

compatibility. Our custom fine-tuning script assembled batches of images and corresponding binary masks, applying random foreground points as prompts for each object instance. Images and masks were uniformly rescaled to a maximum size of 1024 pixels to standardize input shapes. The model's mask decoder, prompt encoder, and image encoder were all set to trainable.

Note: the data loading pipeline, custom PyTorch `Dataset` class, and fine-tuning loop, including batching, augmentation, prompt sampling, custom collate function, explicit mask preparation, and IoU-based loss calculation, were all self-implemented. These components directly handle ClearPose image/mask conversion, prompt selection, dataset batching, and training/validation epoch management, rather than relying on pre-packaged SAM2 fine-tuning scripts.

Fine-tuning was performed over multiple sessions, with the training loop tracking both loss using binary cross-entropy and Intersection over Union at the batch and epoch level. For the first 10 epochs, we used the Adam Optimizer with a learning rate of 1×10^{-3}, and for the next 5 epochs, we used a learning rate of 1×10^{-3}. Across the total training run of 15 epochs, the model achieved a peak average IoU of 76% on the validation set, with clear improvement over the initial model's performance. This demonstrates that SAM2 can be successfully adapted to the specific visual statistics of transparent objects in a realistic robotic setting.

2) Hybrid Pipeline: YOLOv8 Detection plus SAM2 Segmentation: While prompt-based fine-tuning im-

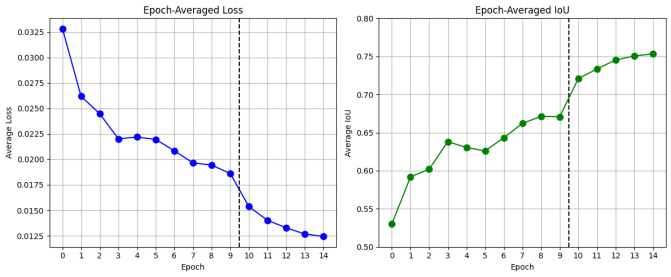

Fig. 7. SAM2 epoch-averaged Loss and IoU

proved classical segmentation accuracy, further qualitative experiments confirmed that the strongest results were achieved through a hybrid pipeline: bounding boxes generated by our YOLOv8 detector were used as bounding-box prompts for SAM2. Each detected bounding box was passed to SAM2's predictor to focus segmentation on a specific candidate region, thus constraining the search space and suppressing spurious background activations.

This approach yielded segmentation masks that adhered closely to object boundaries, and most transparent objects in a given scene were correctly individuated. Visual inspection across a range of test images confirmed that this combined method outperformed SAM2 alone. Typical failure cases included slight over-segmentation when objects had busy internal patterns, or under-segmentation along faint boundaries; however, these cases were infrequent.

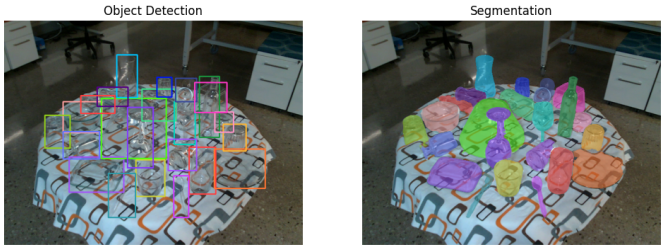

Fig. 8. Detection with YOLOv8 and Prompted-Segmentation with SAM2

C. Object Tracking Across Views

1) Initial DeepSORT Baseline: To track transparent objects across video frames, the first approach implemented was DeepSORT, a widely-used object tracking framework. However, this method exhibited persistent failure modes when applied to our scenes: because the underlying object detector could intermittently miss detections (especially when transparent objects changed appearance or became partially occluded due to camera shakes and blurs), DeepSORT would incorrectly drop or reassign object identities. As a result, tracks were fragmented and long-term correspondences were unreliable for downstream tasks.

2) Self-Implemented Embedding Similarity Tracking: Given these limitations, we designed a custom object-matching pipeline specifically for transparent objects in multi-view scenes. **All components of this tracking algorithm, including feature extraction, cosine similarity calculation, ID assignment and memory, and matching logic, were fully self-implemented in Python. The only exception is the usage of the pre-trained DINOV2 model for object embedding extraction.**

The core idea is as follows:

- For each video frame, the pipeline runs YOLOv8 detection and SAM2 segmentation to generate bounding boxes and instance masks for transparent objects.
- The region within each detected bounding box is cropped and embedded using the DINOV2 visual transformer, resulting in a per-object feature vector.
- For every frame, object embeddings are matched against the set of tracked embeddings from the previous frame with a half-second gap, or 15 frames. If the cosine similarity between a new object and any existing tracked object exceeds a threshold (0.7), the object's ID is carried forward; otherwise, a new identity is assigned.
- This approach is resilient to moderate changes in appearance, shape, and pose—as the DINOV2 features capture high-level visual semantics rather than just mask overlaps or centroids.

3) Results and Qualitative Analysis: Visual inspection demonstrate that this embedding-based tracking system outperformed DeepSORT in our scenarios. The custom tracker reliably maintained object IDs across camera translations, rotations, and cases of moderate appearance changes. These are the scenarios where DeepSORT typically failed due to detector instability. In the test scenes, objects could be consistently tracked as they moved in and out of the field of view, and instance masks remained associated with the correct ID even when shape and appearance changed considerably due to the camera pan.

Occasional ID switches or splits occurred, most often when multiple transparent objects visually overlapped or when multiple similar-looking instances entered the scene simultaneously. However, overall object trajectory

tracking quality was acceptable and better than expected for a self-implemented method.

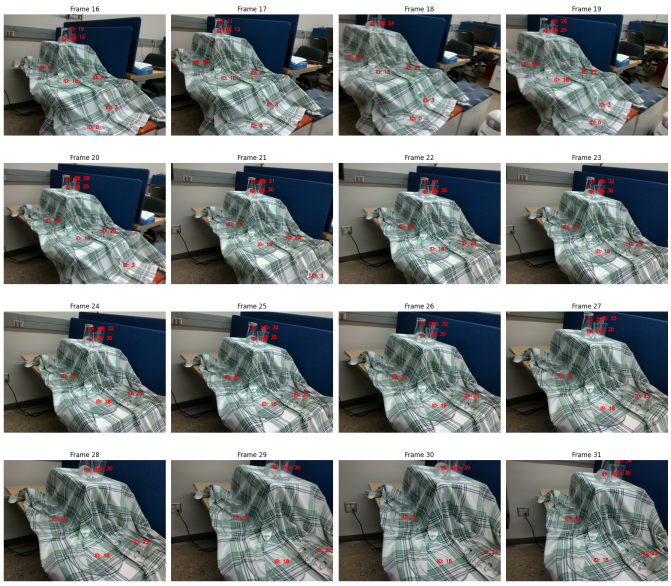

Fig. 9. Object ID matching and tracking on an example scene

D. 3D Segmentation and Pose Estimation

We used VGGSfM to generate a sparse reconstruction of the input scene. Because VGGSfM scales poorly, we sampled images every 15 frames from the start of the video for a total of 32 images. These images were used for both our 2D object detection/segmentation and our 3D reconstruction. Qualitatively, VGGSfM seemed to perform well on the dataset, successfully reconstructing most transparent objects (Fig. 10). Note the protrusion

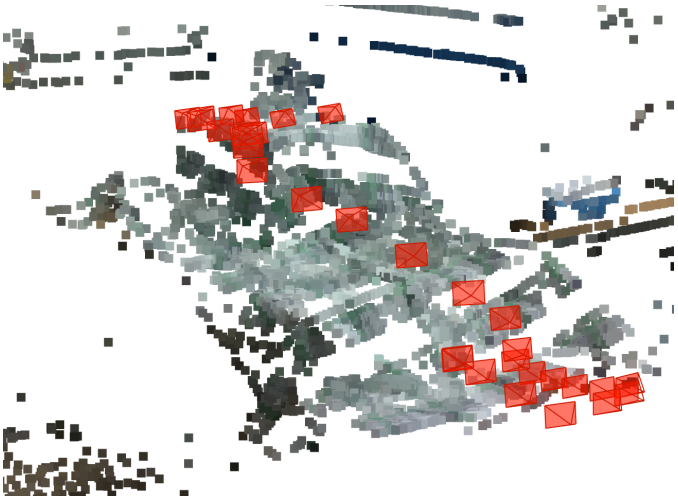

Fig. 10. VGGSfM output

of points above the top of the main tablecloth surface. However, having only a small number of images meant our set of input views was not diverse.

We projected the 2D segmentation masks onto the 3D points, considering only points masked in 80% of relevant camera views to be part of the 3D segmentation. We combined two object IDs if the number of points shared was more than 50% of the number of points corresponding to each object ID. We eliminated all objects that were detected in fewer than 3 views.

This approach was mostly successful. Compared to 9 total objects in the scene, our approach detected and masked 10 objects. The tenth object was a duplicate of the rectangular container on the left which was not caught by the deduplication pass (Fig. 11).

1) 6D Pose Estimation: We manually assigned 3D object point clouds from the ClearPose dataset to their corresponding point clouds, then ran Scale-ICP from 10 different initializations on each object. Results are displayed in Fig. 12, with estimated object poses shown in white.

Results were mixed, with several main failure modes. One failure mode results from insufficient geometry. For instance, the mug (orange points) has complicated geometry that VGGSfM did not faithfully reconstruct. Therefore, results of Scale-ICP were inconsistent. The wine glass also had complicated geometry that failed to reconstruct. Because Scale-ICP has an additional degree of freedom, it is especially vulnerable to local minima where the scale factor converges to an incorrect value.

Another failure mode results from outliers. Due to the limited number of images, our segmentation mask processing still includes false positive points from the background that should be occluded. The rectangular container (blue/green points, left) fails due to this problem, as its 3D segmentation erroneously includes many background points, causing the scale factor to explode. Similarly, one of the water cup point clouds has an extremely large scale factor as its segmentation includes background points that are very far away. This problem could potentially be remedied by a more robust variant of ICP with some level of outlier detection.

A final failure mode occurs due to occlusion. One of the objects in this scene is a plastic fork inside a cup. The 2D segmentation process only includes the head of the fork in the mask, as the remainder is occluded by the cup. Therefore, its pose cannot be reconstructed, as its 3D segmentation is much smaller than it should be.

However, several object poses were reconstructed accurately. The pose of the plate (brown points) was

successfully reconstructed. To reliably reconstruct the plate, using different initializations was necessary, as some initializations led to the plate being reconstructed upside down. The pose of the lower bottle (blue points) was also successfully reconstructed due to the different initializations.

2) Quantitative Results: Quantitative results are difficult to obtain for 6D pose estimation for a number of reasons. First, the 3D reconstructions produced by VGGSfM have different scaling than those used in ClearPose, so manual tuning of the scaling parameter is necessary to align the "true" object poses to begin with. Second, due to rotational symmetry, we cannot measure the difference in rotation matrices, since different rotations could easily be essentially equivalent. Third, due to randomness in the 3D reconstruction, the position of the first camera (which we use to map the ClearPose ground truth poses to our reconstruction) varies slightly, so the "true" object poses visually appear to be slightly offset. Finally, we have no particular approach to compare against and no training process.

Therefore, we compare the Intersection over Union (IoU) of the minimal bounding boxes containing each object with its "true" pose and with its estimated pose:

TABLE I
IOU FOR ALL OBJECTS' 3D BOUNDING BOXES

Object	IoU
Lower bottle	0.3911
Mug	0.2446
Square container	0.0299
Wine glass	0.2522
Cup with fork	0.004
Plate	0.5749
Tipped over cup	0.2166
Fork	0.0010
Back cup	0.0

These results mostly reflect the effect of outliers on each object. The exception is the wine glass, which is in the correct position but has a completely incorrect orientation.

V. CONCLUSIONS

Our results show that a hybrid deep learning strategy can effectively address transparent object identification, segmentation, and tracking. By combining YOLOv8 for object detection and SAM2 for segmentation, we achieved accurate instance segmentation of transparent objects, even in cluttered scenes. SAM2's segmentation performance was greatly improved by the use of bounding-box prompting, and fine-tuning of the model itself.

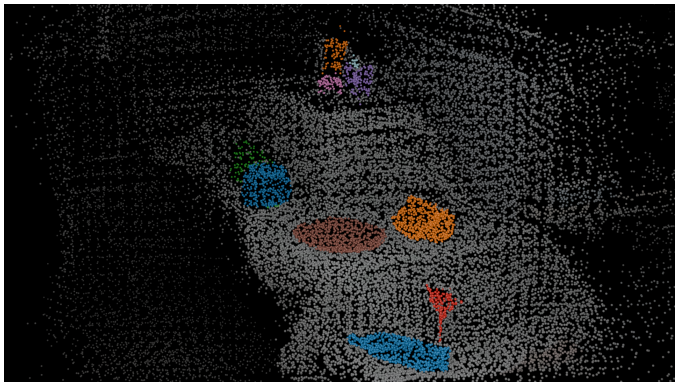

Fig. 11. 3D Segmented Points

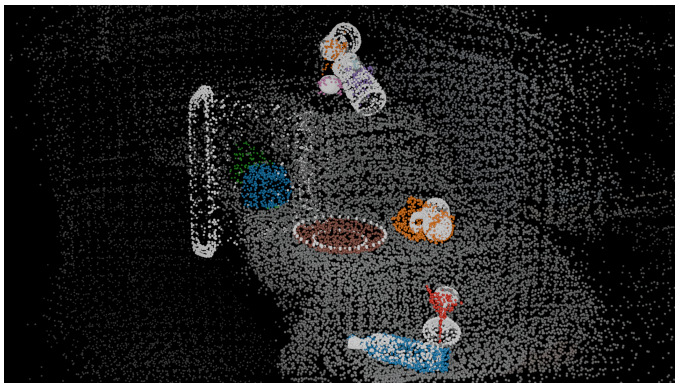

Fig. 12. 3D Segmented Points with Estimated Posed Objects

For object tracking across views, we discovered traditional methods like DeepSORT struggled with the unique challenges posed by transparent objects, such as intermittent occlusions and appearance changes. Rather, our self-implemented embedding-based tracker offered more consistent tracking throughout camera movements by utilizing DINOV2 characteristics for object matching.

For 3D segmentation and 6D pose estimation, we discovered that traditional methods can work, but they depend heavily on the accuracy of the 3D reconstruction. While VGGSfM did reconstruct some geometry from the transparent objects, it did not reconstruct it accurately enough for ICP to estimate most of the poses.

MEMBER CONTRIBUTIONS

- **Lex Byrne**: SfM + 3D reconstruction, 6D pose estimation, report writing.
- **Vinh Nguyen**: Object detection + segmentation, object tracking, report writing.

REFERENCES

[1] S. Sajjan, M. Moore, M. Pan, G. Nagaraja, J. Lee, A. Zeng, and S. Song, "Clear grasp: 3d shape estimation of transparent objects for manipulation," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 3634–3642

[2] X. Chen, H. Zhang, Z. Yu, A. Opipari, and O. Chadwicke Jenkins, "Clear-pose: Large-scale transparent object dataset and benchmark," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 381–396.

[3] I. Ihrke, K. N. Kutulakos, H. P. A. Lensch, M. Magnor, and W. Heidrich. Transparent and specular object reconstruction. *Computer Graphics Forum*, 29(8):2400–2426, 2010

[4] E. Xie, W. Wang, W. Wang, P. Sun, H. Xu, D. Liang, and P. Luo, "Trans2Seg: Transparent Object Segmentation with Transformer," 2021.

[5] H. Mei, X. Yang, Y. Wang, Y. Liu, S. He, Q. Zhang, X. Wei, and R. W. Lau, "Don't hit me! glass detection in real-world scenes," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 3687–3696.

[6] W. Deng, D. Campbell, C. Sun, S. Kanitkar, M. Shaffer, and S. Gould, "Differentiable Neural Surface Refinement for Modeling Transparent Objects," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 20268-20277

[7] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Radle, C. Rolland, L. Gustafson et al., "Sam 2: Segment anything in images and videos," arXiv preprint arXiv:2408.00714, 2024.

[8] J. Wang, N. Karaev, C. Rupprecht, and D. Novotny, "VGGSfM: Visual Geometry Grounded Deep Structure From Motion," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 21686-21697.

[9] Z. Li, Y. Yeh, and M. Chandraker, "Through the looking glass: Neural 3d reconstruction of transparent shapes," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1262–1271, 2020.

[10] Y. Yao, Z. Luo, S. Li, T. Fang, and L. Quan, "Mvsnet: Depth inference for unstructured multi-view stereo," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 767-783.

[11] J. Ichnowski, Y. Avigal, J. Kerr, and K. Goldberg, "Dex-NeRF: Using a neural radiance field to grasp transparent objects," in *Conference on Robot Learning (CoRL)*, 2020.

[12] P. Lai and C, Fuh, "Transparent object detection using regions with convolutional neural network," in *IPPR Conference on Computer Vision, Graphics, and Image Processing*, 2015, pp 1-8.

[13] Y. Zhang and T. Funkhouser, "Deep depth completion of a single rgb-d image," in *IEEE Con- ference on Computer Vision and Pattern Recognition (CVPR)*, 2018

[14] S. Ying, J. Pend, S. Du, and H. Qiao, "A Scale Stretch Method Based on ICP for 3D Data Registration," in *IEEE Transactions on Automation Science and Engineering*, 2009